

Module 8: Week 10: Working with React – Part 2

Module Overview

- HTML events refer to user actions such as clicks, keypresses, or mouse movements that can trigger specific browser behaviours.
- React uses a camelCase syntax for event attributes and requires passing function references instead of strings to handle events.
- Event handling in React allows nested functions within components, enabling dynamic response to user interactions like button clicks.
- Text input changes are captured using the onChange event and accessed through event.target.value to reflect real-time user input.



Handle updates or changes to the value of an input field, such as a textbox



Access the event object via the specified argument (e.g., event)



Retrieve the typed value using event.target.value

```
function EchoComponent() {  
  function printContent(event) {  
    console.log(event.target.value);  
  }  
  
  return <input type="text" onChange={printContent} />;  
}
```

- Form submissions are handled using the onSubmit event, often combined with event.preventDefault() to prevent default page reloads.



Handles form submissions events in React



Trigger actions like network requests or data processing on submit



Call the handleSubmit function when the form is submitted

```
function MyFormComponent() {  
  function handleSubmit(event) {  
    event.preventDefault();  
    alert("form submit");  
    //perform network request programmatically  
  }  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input type="text" />  
      <br />  
      <input type="submit" value="Submit" />  
    </form>  
  );  
}
```

```
function MyFormComponent() {  
  function handleSubmit(event) {  
    event.preventDefault();  
    ...  
  }  
  ...  
}
```

Working with React Hooks and States

- Props are immutable values passed from parent to child components, while state holds data that can change over a component's lifecycle.

State management required ES6 class components initially

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    //initial state
    this.state = { textColor: "#4ad033" };
  }
}
```

- React automatically re-renders components when state changes, ensuring the user interface remains in sync with underlying data.

```
class MyComponent extends React.Component {
  ...
  updateColor(event) {
    this.setState({
      textColor: event.target.value
    })
  }
  ...
}
```

- Hooks enable function components to use state and other React features without converting them into class components.



React hooks, like `useState()` and `useEffect()`, manage state and side effects.

```
import { useState } from "react";
```

```
import XXX from "./XXX"; (notice ./)
```

```
import { XXX } from "XXX"; (notice no ./)
```

- The useState() Hook returns a state variable and a setter function, typically accessed using array destructuring for cleaner syntax.

```
import { useState } from "react";
```

```
import { useState } from "react";

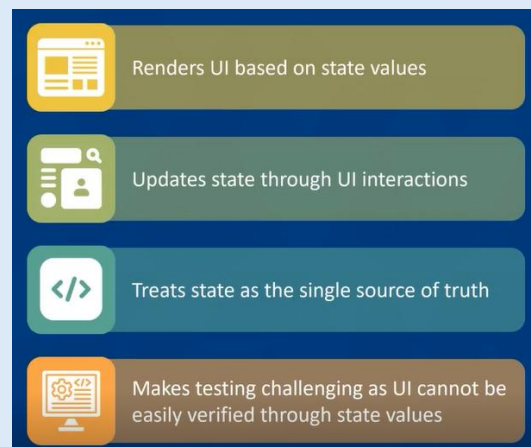
function App() {
  //initialize a state hook
  const [count, setCount] = useState(0);
}
```

```
import { useState } from "react";
function App() {
  //initialize a state hook
  //count is a variable containing the current value of the count
  //setCount is "setter" function for modifying the value of count
  const [count, setCount] = useState(0);
  ...
}
```

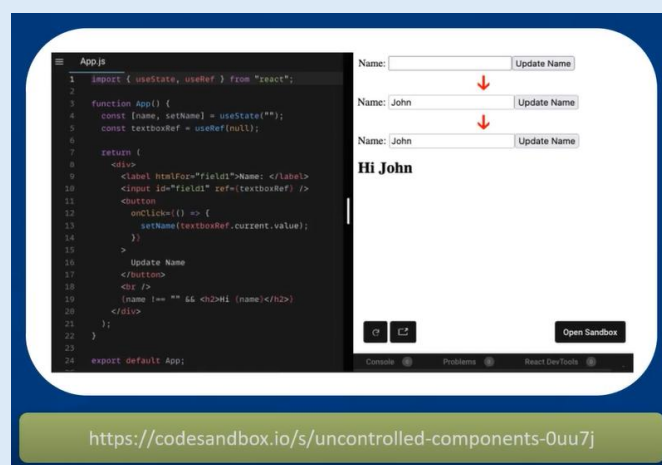
- Improper state updates without conditions or events can cause infinite re-rendering, making controlled state management essential in React.

Controlled vs Uncontrolled Components

- Controlled components rely on React state to manage form inputs, ensuring the state fully determines the user interface at all times.
- State updates occur with every input change in controlled components, making them easier to test and debug using state inspection.



- Uncontrolled components allow the DOM to handle input values, updating state only during specific events such as button clicks.



- Dependence on the DOM in uncontrolled components introduces inconsistencies between the UI and the internal application state.
- Controlled components are generally preferred for consistency, reliability, and better compatibility with unit testing in React applications.

Asynchronous JavaScript and XML (AJAX)

- JSON is a lightweight, text-based data exchange format commonly used to transmit structured data between clients and servers.
- AJAX enables asynchronous communication between a web page and a server, allowing dynamic updates without reloading the entire page.

```
const jsonString = '{ "name": "John Doe", "member": true, "age": 25}';
const obj = JSON.parse(jsonString); //get JS objects from JSON string
```

```
const obj = [
  { name: "jane", age: 20 },
  { name: "tom", age: 24 },
];

//convert from a JS object to a JSON string
//['{"name":"jane","age":20}, {"name":"tom","age":24}']
const json = JSON.stringify(obj);
```

- JSON syntax resembles JavaScript object literals but requires double quotes around property names to maintain valid formatting.
- RESTful APIs are widely used in modern applications to deliver structured JSON responses through AJAX requests initiated by the client.
- The `fetch()` function offers a modern, promise-based approach for making AJAX requests, replacing the older and more complex `XMLHttpRequest`.

Returns a Promise that resolves to an object representing the response to the request

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then((res) => res.json())
  .then((json) => {
    console.log(json);
    //object:
    //{id: 1, name: "Leanne Graham", ... }
  });
```

Working with useEffect Hooks

- The `useEffect` Hook ensures side effects such as data fetching are executed only when necessary, improving application performance.
- Uncontrolled fetch logic inside function components leads to repeated execution during re-rendering, resulting in unnecessary server communication.



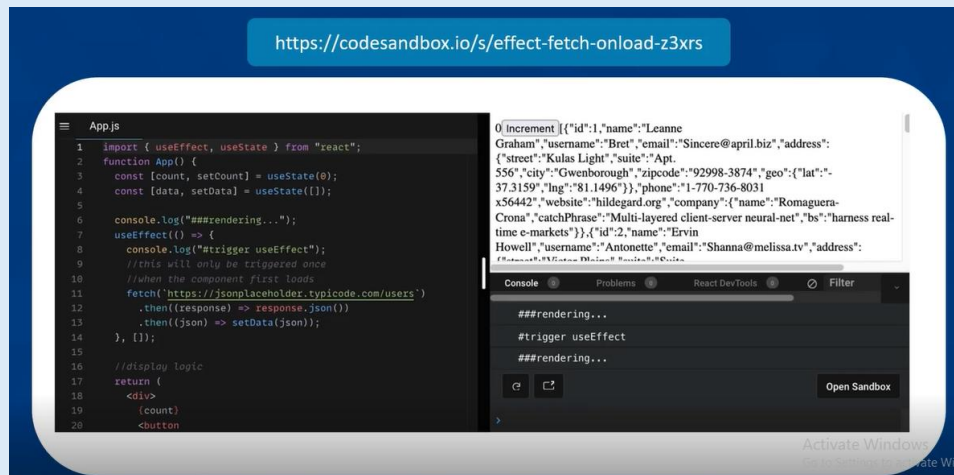
To run fetch logic only when needed, we use `'useEffect'`, imported from React

```
import { useEffect } from "react";

function App() {
  useEffect(() => {
    //fetch data logic
    fetch('https://jsonplaceholder.typicode.com/users?_limit=${size}')
      .then((response) => response.json())
      .then((json) => setData(json));
  }, [size]); //size - dependency array that depends on the "size" variable
  ...
}
```

- The dependency array in `useEffect` provides control over when the effect executes, based on changes to specified variables.

```
function App() {
  ...
  useEffect(() => {
    //fetch data logic
    fetch('https://jsonplaceholder.typicode.com/users')
      .then((response) => response.json())
      .then((json) => setData(json));
  }, []);
  ...
}
```



- An empty dependency array in `useEffect` guarantees that the effect runs only once during the initial render of the component.
- Use of multiple `useEffect` Hooks in a single component allows separation of concerns and clearer handling of different side effects.

Working with `useRef` and Rules of Hooks

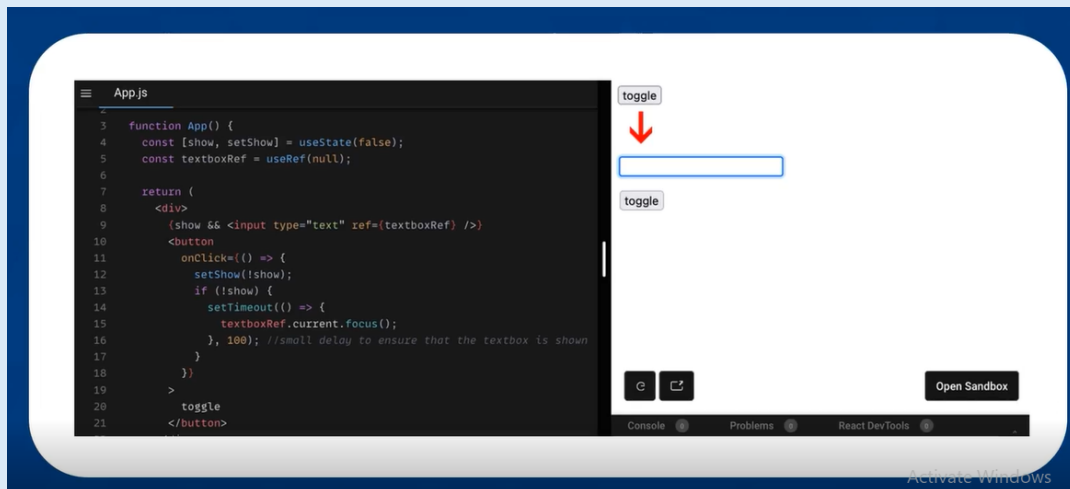
- The `useRef` hook enables referencing HTML elements and storing mutable values across renders without triggering re-renders, improving performance and interactivity.

Useful for referencing HTML elements, such as for animations

```
import { useRef } from "react";

function App() {
  //set initial value as null
  const textboxRef = useRef(null);
  ...
  <input type="text" ref={textboxRef} />
}
```

- DOM access via `.current` allows direct manipulation, such as focusing input fields after conditional rendering and delayed execution for smoother user experience.



- State updates in React occur after function execution, requiring careful timing when combining useRef with conditional rendering and asynchronous logic.
- Mutable variables created with useRef persist across renders, making them ideal for storing non-UI data like timers or previous state values.
- React hook rules mandate calling hooks only at the top level of functional components, avoiding loops, conditions or nested functions to ensure predictable behaviour.

Call Hooks only at the top level, not inside loops, conditions, or nested functions

Call Hooks only from React functions, not regular JavaScript functions

<https://react.dev/reference/rules/rules-of-hooks>

```
//BAD!
if (condition) {
  useEffect(()=>{ ... });
}
```

React Router

- React Router enables single-page applications to simulate multi-page navigation by dynamically rendering components based on the URL path.
- HashHistory and BrowserHistory are the two routing mechanisms available in React, each with distinct URL behaviours.

Frontend routing determines components based on the URL path.

HashHistory uses URLs with a hash symbol to render components.

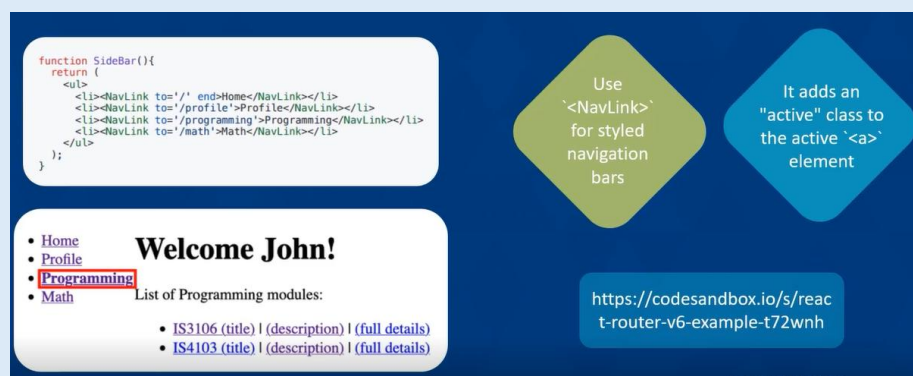
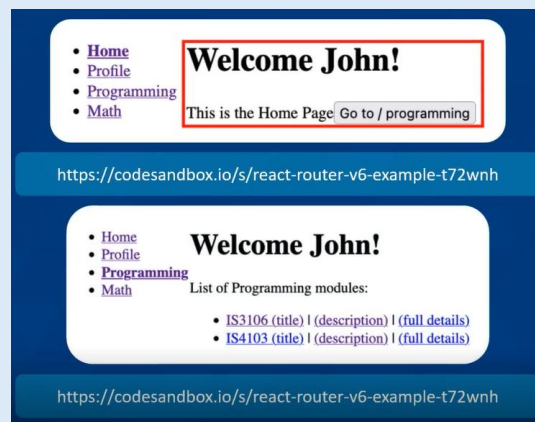
<http://domain/#/path>

BrowserHistory uses backend-routing URLs to display components.

<http://domain/admin/login>

<http://domain/index.html>

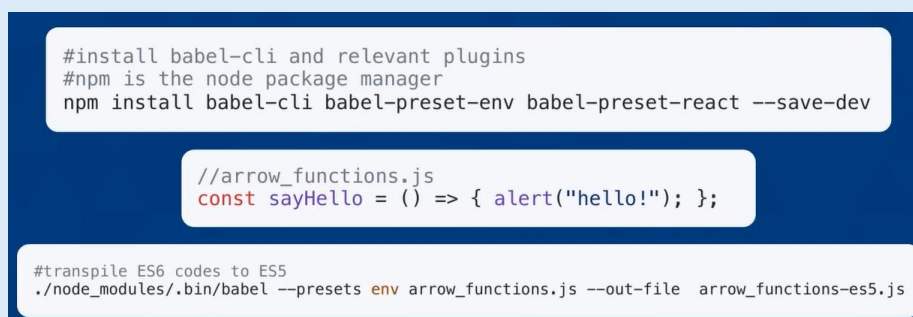
- Wrapping the App component with either HashRouter or BrowserRouter allows routing setup, and Route and Routes control which components display for specific paths.
- Navigation between pages is achieved using Link or NavLink components, where NavLink adds styling to highlight the active link.



- Dynamic routing with route parameters is handled using useParams, and programmatic navigation is possible through the useNavigate hook.

Create React App

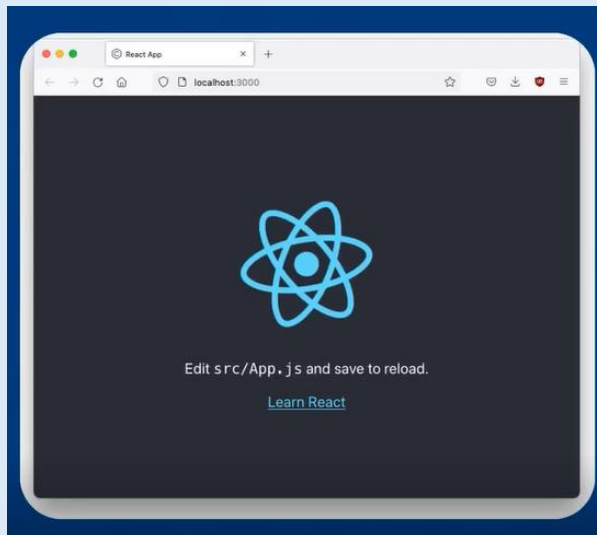
- Create React App is a tool that simplifies the setup of production-ready React applications with minimal configuration.
- Transpiling is essential for browser compatibility and Create React App automates this process using tools like Babel in the background.
- Manual transpilation with Babel CLI is possible but inefficient, especially during development where frequent changes occur.



- Create React App offers a streamlined development experience, including auto-reloading and bundling for deployment.

```
#create a new app (e.g. new-app)
#npm will check whether create-react-app is installed
#and download and install if not
#npm will also use the latest version of react
npm create-react-app new-app
cd new-app
```

```
#start the development server
npm start
```



- The tool uses a familiar file structure and works well with editors like Visual Studio Code for efficient local development.

Module Summary

- Foundational understanding of React's architecture, including component-based design, Virtual DOM, JSX syntax, and the role of transpilation.
- Modular coding practices using ES6 modules and props to enable reusable, maintainable, and dynamic user interface components.
- Efficient coding in React using modern ES6 features such as arrow functions, template literals, rest/spread operators, and destructuring.



React Props enable data sharing between components, enhancing reusability and adaptability.



The ES6 Module System improves organisation by separating components into individual files for export and import.



CodeSandbox offers a hassle-free development environment.



Modern ES6 constructs like arrow functions, template strings, object literal shorthand, rest and spread operators, and destructuring simplify code and are widely used in React.

- Practical skills in managing state and side effects using React hooks and distinguishing between controlled and uncontrolled components.
- Complete workflow for building and deploying React apps using React Router for navigation and Create React App for development setup.

THE END